

RCS Enterprise Modern - Piping Script Manual

This manual provides a comprehensive guide to formatting, writing, and executing **Piping Scripts** within the software. The pipe Scripting Engine uses strict Civil-style linework directives to build and validate intelligent piping networks automatically.

1. Engine Toggles

You can write mixed scripts (COGO + Piping). To ensure the compiler processes piping sections correctly, use the engine toggles:

- **PIPE-ENGINE-ON** : Starts processing piping scripts.
- **PIPE-ENGINE-OFF** : Pauses piping processing (useful for intermixing pure COGO code).

Note: Standard COGO commands like NEZ, PT, CLEAR, and RESET are recognized unconditionally so they do not error the pipe engine.

2. Pipe Runs (PRUN)

All piping connectivity must occur within an active Pipe Run (PRUN) context segment. A single pipe run defines the shared physical properties of the upcoming pipes.

Start a Pipe Run

Syntax: PRUN START <UtilityCode> DIAM <Size> MAT <MaterialName> **Example:** PRUN START W DIAM 8 MAT PVC

* **UtilityCode:** The abbreviation for your system (e.g., W for Water, WW for Waste Water, R for Reclaimed). *

DIAM: The numeric diameter of the pipe. * **MAT:** The material string. *Must match exactly with the Materials Validation Database (e.g. PVC, DIP).*

End a Pipe Run

Syntax: PRUN END Closes the active run. You must close a run before starting a new one.

3. Linework Directives (Connectivity)

Inside a PRUN block, you use **Feature Directives** to connect COGO nodes together. These commands physically map out the system.

(In the examples below, [UTIL] refers to the Utility Code defined in the PRUN block, e.g., W, WW, SS)

Begin (-B)

Syntax: [UTIL]-B <NodeID> **Example:** W-B 102 * Begins a new piping path at the specified Node ID. Automatically stores a structural point here based on your Utility Code.

Continue (-C)

Syntax: [UTIL]-C <NodeID> [OptionalStructureType] **Example:** W-C 103 (Standard) **Example:** W-C 103 Valve (Explicit) * Draws a pipe from the *previously defined* node to this new node. * If you supply an optional string at the end, it overrides the default structural component at that node.

End (-E)

Syntax: [UTIL]-E **Example:** W-E * Explicitly ends the contiguous drawing path. You must use -B again to start a new branch.

Close (-CLS)

Syntax: [UTIL]-CLS **Example:** W-CLS * Closes the figure by drawing a pipe from the current node back to the *start vertex* of this specific path.

4. Structures & Direct Data

Sometimes you need to declare a structure *without* drawing a pipe connected to it, or you want to define invert elevations explicitly.

Store Structure (SS-C)

Syntax: SS-C <NodeID> <StructureCode> **Example:** SS-C 35 Manhole * Manually places a structural component at the given Node ID.

Numeric Segments (Inverts & Math)

Syntax: <FromNode> <ToNode> [InvertStart] [InvertEnd] **Example:** 10 11 12.5 10.2 * If you just list two numbers (or four) inside a PRUN block, the engine connects them directly. * *Note on Slopes:* The system calculates fall mathematically. If it is a Gravity system (WW), the compiler throws validation errors if the calculated slope between the two inverts drops below **0.40%**.

5. Putting It All Together (Real-World Example)

Here is a snippet showing how to combine these rules into a working script.

```
```text // 1. Generate local geometry NEZ 100 5000.0 5000.0 10.5 NEZ 101 5000.0 5100.0 10.5 NEZ 102 5000.0
5200.0 10.5 NEZ 103 5050.0 5100.0 10.5

// 2. Enable engine PIPE-ENGINE-ON

// 3. Start the Water main run PRUN START W DIAM 12 MAT DIP

// 4. Draw the main line W-B 100 W-C 101 Valve W-C 102 W-E

// 5. Build a branch line off the Valve at 101 W-B 101 W-C 103 Hydrant W-E

// 6. Close the context PRUN END ```
```

## Compilation Checking

When you run a script, the engine strictly maps out the materials and properties. If you declare `MAT PVC` but `PVC` is not imported into your current Master Validation List, the compiler will safely error the script out to prevent database corruption.